

Seeing the Geometry:

Toy Experiments Behind a Learned Delay-Window Anomaly Detector

Benji Clothier

Victoria University of Wellington

Internal note for supervisors, 5 October 2026

This note shows, mostly in pictures, what the learned detector in the DG_AD project does and where it goes wrong. Every number and figure comes from runs in our repositories. Where a result is a diagnosis made after seeing the data rather than a prediction tested beforehand, I say so. The full record, with the predictions written down before each run, is in `DG_AD/docs/PROGRESS.md`.

1 The picture behind both of our earlier detectors

Our earlier detectors cut a time series into overlapping windows of length L and treat each window as a point in $\mathbb{R}^L$. Healthy windows form some set in that space. The global detector models that set by a Gaussian cloud, so in effect by its linear span. The local detector models it by the training windows themselves, and scores a new window by the distance to the nearest one.

Figure 1 shows what this means in two dimensions, with a curved arc of healthy points and three probe points:

- **A** lies well away from the healthy arc;
- **B** lies inside the bowl of the arc: within the span of the healthy data, but off the set;
- **C** lies on the continuation of the curve beyond its last training point. If the curve is the true healthy behaviour, C is healthy but was never seen.

The global detector gives B a score of exactly zero: it cannot see anomalies inside the span. The local detector sees B, but it scores C (0.67) almost as high as B (0.53), so it would raise false alarms on healthy behaviour it has not seen. This is the trade-off we want to get out of.

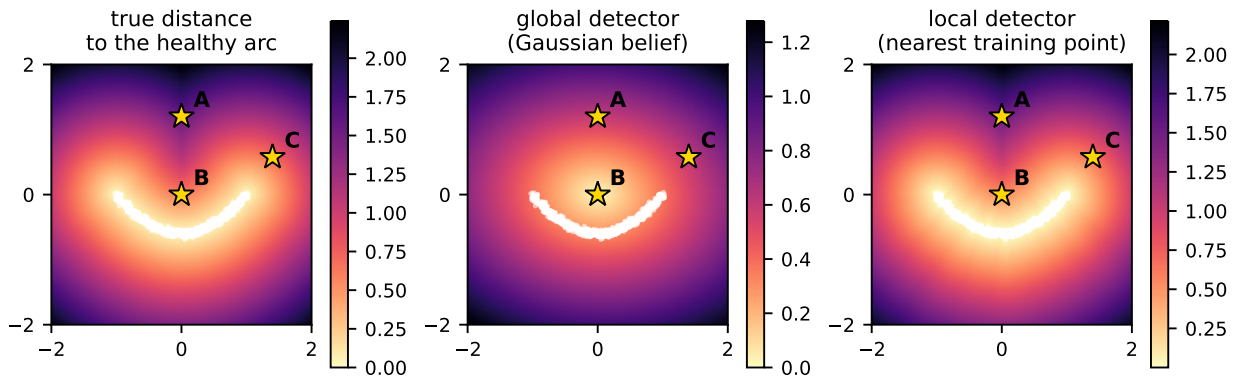


Figure 1: Healthy points (white) on a curved arc, and the score each approach gives every point of the plane (dark = high score). *Left:* the true distance to the arc. *Middle:* the global detector is blind around B, inside the bowl. *Right:* the local detector sees B but also scores C, the unseen continuation of the curve, highly.

2 A network that learns the healthy set

The ideal picture. Suppose we had two perfect tools. The first is a projection $m(y)$ that sends any window y to the nearest point of the healthy set. The second gives, at every point x of the healthy set, the projection $P(x)$ onto the set’s tangent plane there; $P(x)$ has eigenvalue 1 for tangent directions and 0 for orthogonal ones. The anomaly score would then be the orthogonal part of the step from $m(y)$ to y :

$$s(y) = \|(I - P(m(y))) (y - m(y))\|.$$

Figure 2(a) shows this for the simplest healthy set, the line $x_2 = 0$. With a perfect projection, $y - m(y)$ is already orthogonal to the set, so the score is just the distance to the set. The projector matters when m is not perfect (Figure 2(b)). If $m(y)$ is shifted in the tangent direction, $y - m(y)$ gains a tangent part; the projector removes it and leaves the true distance.

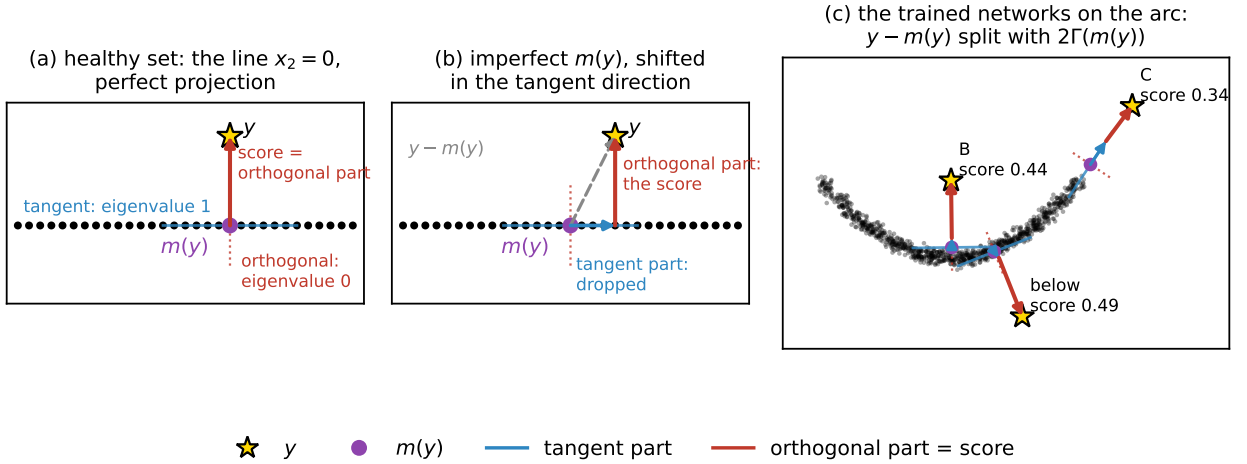


Figure 2: The geometry of the score. (a) The healthy set is the line $x_2 = 0$; $m(y)$ is the nearest point on it; at $m(y)$ the tangent direction has eigenvalue 1 and the orthogonal direction eigenvalue 0; the score is the orthogonal part of $y - m(y)$, here the whole of it. (b) If $m(y)$ is shifted in the tangent direction, $y - m(y)$ has a tangent part (blue), which the projector drops, leaving the orthogonal part (red), the true distance. (c) The trained networks on the curved toy set: for three points y , the network’s $m(y)$ and the split of $y - m(y)$ into tangent and orthogonal parts using the network’s $2\Gamma(m(y))$.

The networks. We learn both tools with Riemannian Metric Matching (Bamberger et al., ICML 2026). Training adds Gaussian noise of a chosen size ε to healthy windows. The **denoiser** m is trained to undo the noise, and approximates the projection. The **metric network** Γ is trained so that 2Γ matches how far, and in which directions, the clean windows behind noisy inputs are scattered around the denoiser’s output. Near a smooth healthy set that scatter lies in the tangent plane and has the size of the noise, so on the set $2\Gamma(x) \approx P(x)$. Neither network needs a neighbour search, and neither is told the dimension of the set. Our score is therefore

$$s_2(y) = \|(I - 2\Gamma(m(y))) (y - m(y))\|,$$

with the projector evaluated at the denoised point. Our first experiments evaluated it at y instead; Section 3 explains the difference.

Where the denoiser is not a projection. Figure 3 shows where the trained denoiser sends each point. Outside the bowl, points go onto the arc, even from far away. There are two departures from a true projection. First, points high inside the bowl, about equally close to both sides of the arc, are sent to a point between the arc’s ends, off the set: the denoiser averages the two candidates instead of choosing one. B, lower down, lands just short of the bottom of the arc, 0.52

away against a true distance of 0.60. This is the “inside the span, off the set” region, and it is a built-in weakness of any score based on $y - m(y)$. Second, past the end of the arc (C), the denoiser carries on along the curve’s continuation, beyond the last training point, instead of stopping at the end point (Figure 2(c)). It extrapolates the curve. Whether that is good depends on whether the continuation is healthy.

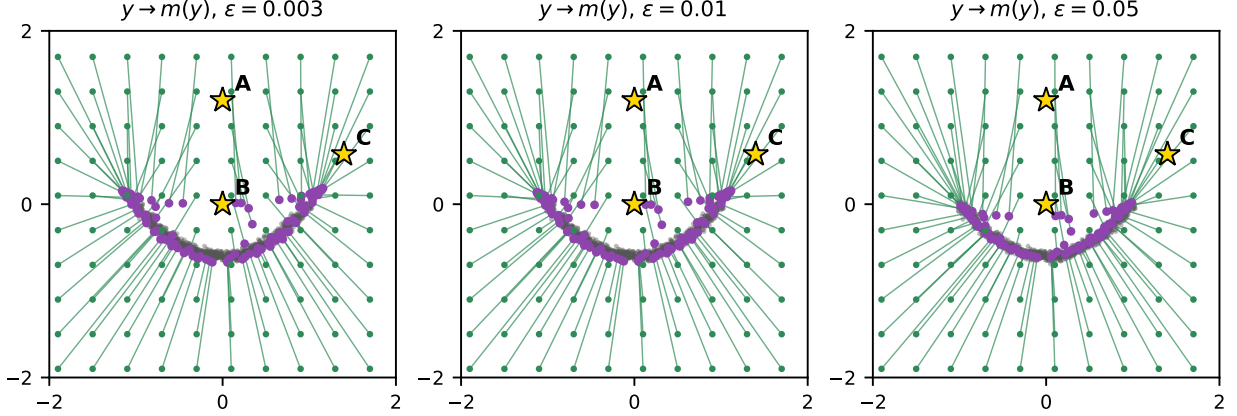


Figure 3: Each line joins a point y (green dot) to the denoiser’s output $m(y)$ (purple dot), at three noise levels ε . Outside the bowl, points are pulled onto the arc. Points high inside the bowl are pulled to the middle, between the arc’s ends.

3 Evaluating the projector at the window itself

Our first experiments used $\|(I - 2\Gamma(y))(y - m(y))\|$, with the projector evaluated at the window y itself. Geometrically this is a proxy. The network’s Γ is defined everywhere, because it was trained on noisy points scattered around the set, and at a point y off the set it returns roughly the tangent projector of the part of the set nearest to y . Close to the set, the two choices agree. Further away, the eigenvalues drift from 1 and 0. Where two parts of the set are about equally near, inside the bowl or between two regimes of a time series, $\Gamma(y)$ blends them, with an eigenvalue *above* 1 in the direction joining them. Appendix A compares the network’s values with the exact ones for this toy. Figure 4 and Table 1 compare the two choices.

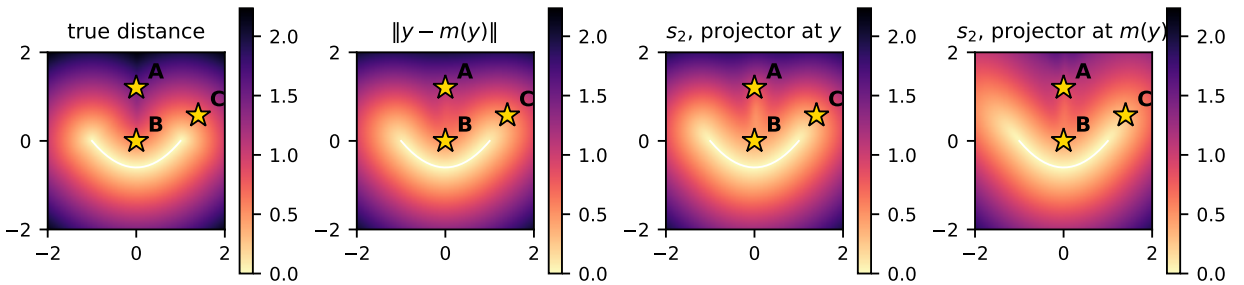


Figure 4: Score maps on one common colour scale (noise level $\varepsilon = 0.01$). From left: the true distance to the arc; the raw denoising step $\|y - m(y)\|$; the learned score with the projector at y ; and with the projector at $m(y)$.

Two things stand out in Table 1. First, every learned score sees B, unlike the global detector, though each under-reads it (0.40–0.52 against a true distance of 0.60), because the denoiser lands B slightly short of the arc, inside the bowl. Second, with the projector at $m(y)$, the score at C drops to 0.34: the tangent direction at the arc’s end points the way the curve continues, so a window that carries on along the curve is treated as more healthy. In this toy, the projector at the denoised

score at probe	A (far away)	B (inside the bowl)	C (unseen continuation)
true distance to the training arc	1.56	0.60	0.70
global detector	0.60	0.00	0.61
local detector	1.51	0.53	0.67
$\ y - m(y)\ $	1.22	0.52	0.56
s_2 , projector at y	0.95	0.40	0.49
s_2 , projector at $m(y)$	0.95	0.44	0.34

Table 1: Scores at the three probe points ($\varepsilon = 0.01$; the scores have different scales, so compare within a row of the same score, not across rows).

point extrapolates in the tangent direction the way the global detector extrapolates within its span, which is the behaviour we wanted from a belief in between. This is one toy, with one noise level; Section 6 tests it on time series.

4 Does the network get the tangent planes right?

Before building a detector on it, we checked the metric network on shapes whose tangent planes are known: a circle, a sphere and a flat torus placed in $\mathbb{R}^{20}$, with noise, and four harder shapes (crossing planes, a plane crossed by a line, a Swiss roll and an unevenly sampled sphere; all defined and drawn in Appendix B). We compared it with local PCA, which was given the true dimension and a neighbourhood size tuned on the truth (both advantages).

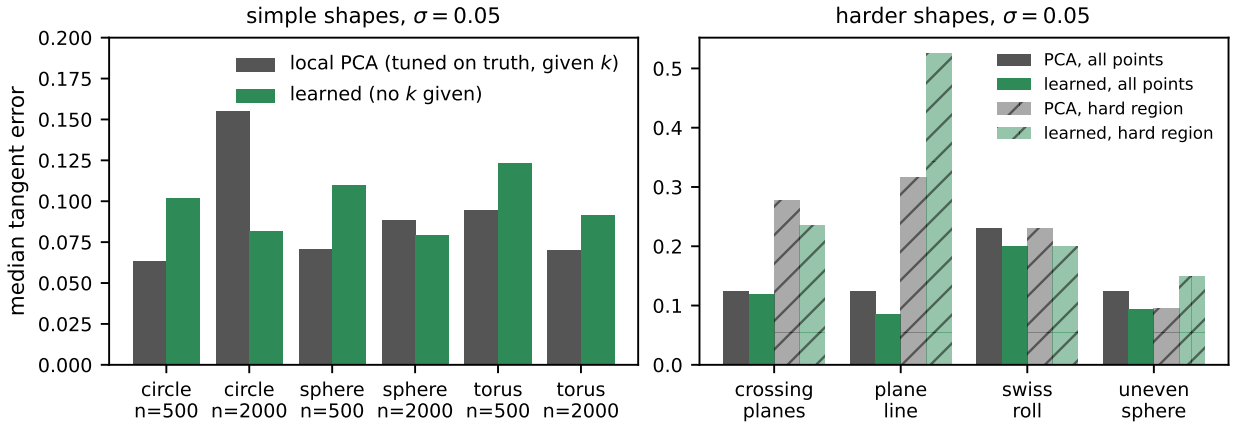


Figure 5: Median tangent-plane error (0 = perfect) at noise $\sigma = 0.05$, median over two seeds; the shapes are defined in Appendix B. *Left:* simple shapes. *Right:* harder shapes. “All points” is the median over every training point; “hard region” is the median over the part of each shape where tangent planes are hardest to estimate: within 0.2 of the planes’ shared line (19% of points), within 0.2 of the point where the line crosses the plane (5.5%, about 110 points, so this median is noisier), the sparse southern half of the sphere (13%), and every point of the Swiss roll, whose two bars are therefore identical by construction. Local PCA (grey) is scored on the same points.

The learned tangents are about as accurate as tuned local PCA, not better (left panel), and the network read off the correct dimension at every point. The Riemannian Metric Matching paper (Bamberger et al., ICML 2026, §5.1) reports “46% improved performance” in tangent-space error relative to a nearest-neighbour carré du champ estimator, for fewer than 4 million points sampled on an 8-dimensional sphere in $\mathbb{R}^{64}$. That is a reference figure, which we have not reproduced. It differs from our test in three ways: a different baseline (theirs is not local PCA), a much higher intrinsic dimension, and no added noise mentioned. Our shapes have dimension 1 or 2, where local PCA is at its strongest, so parity here does not contradict it. On harder shapes, with one noise level used everywhere, the network helps where noise dominates and hurts where the structure changes

abruptly, for example where a line meets a plane (right panel). Whether a noise level chosen point by point removes that weakness is untested. My first attempt at this, a simplified version of the method, failed badly; implemented as the paper describes, it works.

5 Curvature from the same networks

What this section does, and why. The previous sections used the networks to find, at each point of the healthy set, its *tangent* directions: the directions in which the set extends locally. This section goes one order further and estimates how the set *bends* away from its tangent plane, its curvature, using the same two networks and no extra training. We care for three reasons.

- **Curvature limits tangents.** It says how far a straight step in a tangent direction stays close to the healthy set, and so how far the tangent plane can be trusted.
- **Curvature biases the denoiser.** Our earlier kernel-residual work showed that the denoising step is pulled towards the inside of a curved set, in proportion to its mean curvature. That bias matters for any score built on $y - m(y)$.
- **Curvature could be useful in its own right,** for example as a feature that changes when the healthy behaviour changes. Whether it helps detection has not been tested.

Two classical objects make “tangent directions” and “bending” precise.

- The **first fundamental form** measures lengths of tangent vectors. We need it to turn a velocity into a unit direction.
- The **second fundamental form** measures bending; it is the quantity we estimate.

Together they give the normal curvature in the direction of a tangent vector v ,

$$\kappa = \frac{\|\Pi(v, v)\|}{\mathbf{I}(v, v)},$$

and our estimator below is a finite-difference version of exactly this ratio.

Background: the first and second fundamental forms. Let M be a smooth set (a curve, surface or higher-dimensional manifold) in $\mathbb{R}^D$. Let x be a point of M , with tangent plane $T_x M$ and tangent projector P .

- The **first fundamental form** measures lengths and angles of tangent directions: for tangent vectors $w, w' \in T_x M$, $\mathbf{I}(w, w') = \langle w, w' \rangle$, the ordinary dot product restricted to the tangent plane. It says nothing about bending: a flat sheet and the same sheet rolled into a cylinder have the same first fundamental form.
- The **second fundamental form** measures bending. Take any curve γ on M with $\gamma(0) = x$ and velocity $\gamma'(0) = v$; the velocity v is a tangent vector at x , and $u = v/\|v\|$ is its unit direction. Its acceleration has a tangent part, which depends on how the curve is drawn within M , and an orthogonal part, which M forces on any curve that stays on it:

$$\Pi(v, v) = (I - P) \gamma''(0).$$

$\Pi(v, v)$ is a vector in the orthogonal directions. For a surface in $\mathbb{R}^3$ it is a number times the normal. It extends to a symmetric bilinear form $\Pi(w, w')$ on pairs of tangent vectors.

Three properties matter here.

1. $\Pi(v, v)$ depends only on v , not on which curve is used (Meusnier’s theorem).

2. It is quadratic in the velocity, $\Pi(cv, cv) = c^2 \Pi(v, v)$.
3. It is the second-order shape of M . Near p , in tangent coordinates t , the set is approximately the graph

$$\text{orthogonal offset} \approx \frac{1}{2} \Pi(t, t),$$

its best quadratic approximation. This is what local quadratic regression fits.

The **normal curvature** in a unit tangent direction u is $\kappa(u) = \|\Pi(u, u)\|$. For example:

- a plane has $\kappa = 0$ in every direction;
- a circle or sphere of radius r has $\kappa = 1/r$, with $\Pi(u, u)$ pointing to the centre;
- a cylinder of radius r has $\kappa = 1/r$ around it and 0 along its axis;
- the saddle $z = (a^2 - b^2)/2$ at its centre bends upward along the a axis ($\Pi = +1$) and downward along the b axis ($\Pi = -1$). Along the diagonal directions $a = \pm b$, halfway between the two axes, it does not bend at all ($\Pi = 0$): $z = 0$ whenever $a = \pm b$, so the surface contains those two straight lines.

The largest and smallest normal curvatures at a point are the principal curvatures. Their average is the mean curvature, the quantity our earlier kernel-residual work showed the denoising step responds to.

The curvature of a set at a point x , in a tangent direction u , measures how fast the set bends away from the straight line $x + su$ as s grows. We estimate it with the same two networks, in three steps.

1. **A straight line.** Take a point x on the healthy set and a unit tangent direction u there, read off the eigenvectors of $2\Gamma(x)$. On a curved set the straight line $x + su$ leaves the set as s grows.
2. **Denoise it.** Send three points of the line back with the denoiser,

$$c(s) = m(x + su), \quad s \in \{-h, 0, h\}, \quad h = \sqrt{\varepsilon}.$$

The points $c(s)$ lie back on (or near) the set, so they trace a short curve through x that follows the set.

3. **Measure how that curve bends.** With finite differences,

$$\hat{v} = \frac{c(h) - c(-h)}{2h}, \quad \hat{a} = \frac{c(h) - 2c(0) + c(-h)}{h^2}, \quad \hat{\kappa} = \frac{\|(I - 2\Gamma(x)) \hat{a}\|}{\|2\Gamma(x) \hat{v}\|^2}.$$

$\hat{v}$ and $\hat{a}$ are finite-difference estimates of the velocity $c'(0)$ and acceleration $c''(0)$ of the denoised curve. $\hat{v}$ lies, approximately, in the tangent plane $T_x M$, so it plays the role of v in the definitions above. Here $\hat{\kappa}$ is the estimated *normal curvature* of the healthy set at x in the direction u : one over the radius of the circle that best fits the set along that direction. A flat set has $\hat{\kappa} = 0$, a circle of radius r has $\hat{\kappa} = 1/r$, and a large value means the set bends sharply. It is measured in inverse units of length, so halving the size of a shape doubles its curvature.

- The **numerator** is how strongly the denoised curve bends away from the straight line: the orthogonal part of its acceleration.
- The **denominator** is its squared speed. Dividing by it makes $\hat{\kappa}$ a property of the set's shape, not of how quickly the curve is traced out.

Curvature depends on the direction u . The saddle bends upward along one axis, downward along the other, and not at all along the diagonals between them; the flat torus has curvature between 0.71 and 1 depending on direction. In $\mathbb{R}^{20}$ there are many orthogonal directions and no single “up”, so we report only the size of the orthogonal acceleration, not its sign. In the terms above, the numerator $(I - 2\Gamma(x))\hat{a}$ estimates $\Pi(v, v)$ for the denoised curve’s velocity $v = c'(0)$. Dividing by the squared length of the velocity’s tangent part, $\|2\Gamma(x)\hat{v}\|^2$, turns it into $\Pi(u, u)$ for the unit direction (by property 2), and $\hat{\kappa}$ estimates its length.

Geometrically, with $\bar{c} = \frac{1}{2}(c(-h) + c(h))$ the midpoint of the chord joining the outer two points,

$$\hat{a} = \frac{2(\bar{c} - c(0))}{h^2},$$

twice the *sagitta* (the gap between the middle point and the chord) divided by h^2 (Figure 6).

On a circle of radius r , the denoised points sit a distance of about $s^2/(2r)$ inside the straight line, so $\hat{a} \approx -\frac{1}{r}n$ with n the outward normal, and $\hat{\kappa} = 1/r$, the circle’s curvature. Only the orthogonal part of the acceleration is used. Its tangent part reflects only how unevenly the denoised points are spaced along the curve, which is not curvature. A published version of this idea (Wang, Kiani & Weber, 2025, under review) uses the whole acceleration, and is therefore biased upward. The step $h = \sqrt{\varepsilon}$ is the scale at which the denoiser sees the set: smaller steps fall below its resolution, and larger ones average over too much of the set.

Where the formula comes from. Each part has a different source.

- **The construction**, denoising a straight tangent line and measuring the curvature of the result by finite differences, is the “geodesic interpolation” estimator of Wang, Kiani & Weber (2025, under review, §3.3). They take the Frenet–Serret curvature of the whole curve.
- **The orthogonal projection and the division by the squared speed** follow from two textbook facts of differential geometry (do Carmo, *Differential Geometry of Curves and Surfaces*, 1976, Ch. 3, “The Geometry of the Gauss Map”; Lee, *Introduction to Riemannian Manifolds*, 2nd ed., 2018, Ch. 8, “Riemannian Submanifolds”, section “The Second Fundamental Form”).
 - The orthogonal part of a curve’s acceleration is the second fundamental form $\Pi(v, v) = \|v\|^2 \Pi(u, u)$, where $u = v/\|v\|$.
 - By Meusnier’s theorem it is the same for every curve on the set with that velocity, whether or not the curve is a geodesic.
- **The velocity and acceleration** are ordinary central finite differences.
- **That 2Γ approximates the tangent projector** is the result of Bamberger et al. (ICML 2026).
- **The step $h = \sqrt{\varepsilon}$ is our own choice.** It was fixed before the experiments, and its justification (the scale at which the denoiser resolves the set) is a heuristic, not a derivation.

We know of no published derivation of the combined estimator. It was checked on shapes with known curvature, below.

Figure 7 shows the same construction with the trained networks on the 2-D toy, for a longer stretch of line.

We tested this on six shapes with known curvature, placed in $\mathbb{R}^{20}$ (Appendix B): a circle, an ellipse (curvature varying eightfold), a sphere, a flat torus (curvature depending on direction), a saddle and a flat plane. The networks were told neither the dimension nor the direction. Two measures were used:

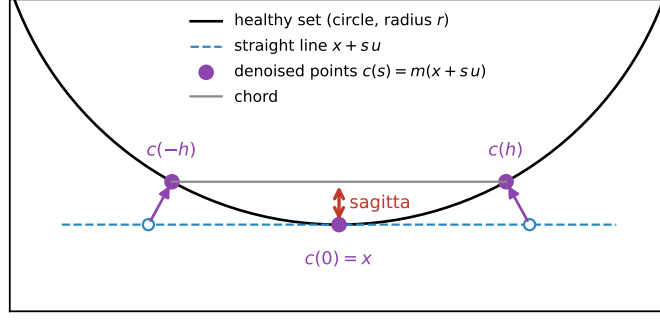


Figure 6: The curvature estimator on a circle, with exact geometry and no network. Three points on the straight tangent line (open circles) are sent back to the set (purple). The gap between the middle point and the chord joining the outer two (the sagitta) gives the acceleration, and hence the curvature. The step h is exaggerated here for visibility; at this step the three-point estimate reads 0.90 instead of 1, and the error shrinks as h does.

- the **median relative error** $|\hat{\kappa} - \kappa|/\kappa$ over 400 test points, which measures size;
- the **rank correlation** (Spearman's ρ), which measures whether the estimate follows how curvature varies across a shape. To compute it, replace the estimated curvatures by their ranks (1 = smallest), do the same for the true ones, and take the ordinary correlation of the two lists of ranks; with no ties, $\rho = 1 - 6 \sum_i d_i^2 / (n(n^2 - 1))$, where d_i is the difference between point i 's two ranks. $\rho = 1$ means the points are in exactly the right order from flattest to most curved, and $\rho = 0$ means no relation. It ignores a constant scale error: if every estimate were 10% too high, ρ would still be 1. It is only defined where the true curvature varies (the ellipse, torus and saddle).

With 2000 training points and noise $\sigma = 0.02$, our targets, set in advance, were met on the circle and sphere (median relative error 0.057 and 0.076) and on the ellipse and saddle (rank correlation with the truth 0.99 and 0.84). They were missed on the torus (0.75 against a target of 0.8) and on the plane (median estimate 0.45 against a target of at most 0.1).

The noise level ε has to be chosen without the ground truth. Before running, we fixed an **automatic, label-free rule**, which uses only the training points.

1. For each ε on the grid $16, 8, \dots, 2^{-9} \approx 0.002$, evaluate $2\Gamma(x, \varepsilon)$ at every training point.
2. Count its eigenvalues above half the largest one; this count is the estimated number of tangent directions $\hat{k}$.
3. Compute the relative gap $(\lambda_{\hat{k}} - \lambda_{\hat{k}+1})/\lambda_1$ between the last tangent eigenvalue and the first orthogonal one.
4. Choose the ε with the largest median gap.

The idea is that at a good scale the tangent eigenvalues (about 1) are cleanly separated from the orthogonal ones (about 0).

Figures 8 and 9 show what went wrong. The estimator itself works over a wide range of noise levels, the plateau $\varepsilon \approx 0.002$ –0.25 in Figure 9, where every curved shape is under the target. On the saddle at $\varepsilon = 0.03$, for example, its median relative error is 0.12 and its rank correlation 0.94. That plateau was identified with the ground truth, so it is not yet a usable rule. Both of its edges have a geometric reason:

- the step $\sqrt{\varepsilon}$ must be larger than the spacing between neighbouring training points, otherwise the denoiser snaps to individual points (2000 points on a unit sphere are about 0.08 apart, hence $\varepsilon \gtrsim 0.006$; this matches the sphere's low bias at $\varepsilon = 0.002$);

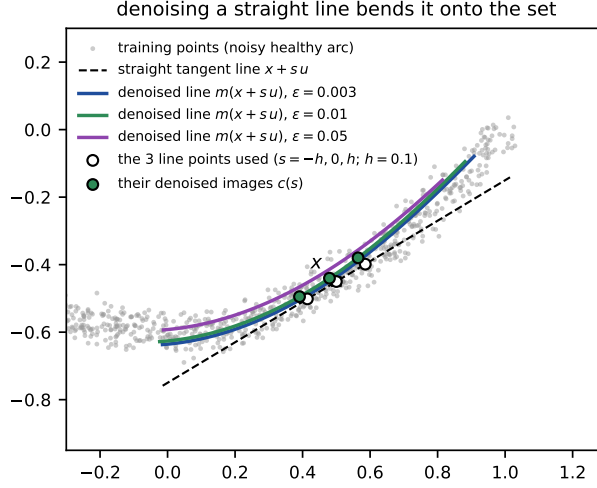


Figure 7: The construction with the trained networks on the 2-D toy. Grey dots: the 1000 noisy training points (the healthy data, not network outputs). Dashed: the straight line $x + su$ through a point x of the arc, along the arc's exact tangent u (in the estimator, u comes from 2Γ). No noise is added to the line. Each coloured curve feeds the line's points to the denoiser $m(\cdot, \varepsilon)$, which treats its input as if it had been corrupted by Gaussian noise of variance ε ; ε is therefore a smoothing scale, not noise on the points. The denoised line follows the arc, so its bending is the arc's bending. At the largest ε it sits slightly inside the bowl, the inward shrinkage that grows with ε . The open circles mark the only three line points the estimator uses at $\varepsilon = 0.01$ ($s = -h, 0, h$ with $h = \sqrt{\varepsilon} = 0.1$), and the filled circles their denoised images. These are slightly shifted and unevenly spaced along the arc, which is why only the orthogonal part of the acceleration is used.

- $\sqrt{\varepsilon}$ must be well below the radius of curvature and the size of the shape, otherwise the denoiser averages over a curved stretch.

In short, point spacing $< \sqrt{\varepsilon} \ll$ radius of curvature. What fails is the automatic rule. On flat or nearly flat shapes, blurring never mixes the tangent and orthogonal directions, so the eigenvalue gap keeps growing with ε and the rule keeps preferring more blur. It chose $\varepsilon = 2$ on this saddle, where the estimate is about ten times too large. The torus miss is narrower: its true curvature only spans 0.71 to 1.0, so a 7% error is enough to cap the rank correlation. The next step is a better rule for ε . Two label-free candidates are: choose the ε at which the estimate changes least between neighbouring grid values, a plateau rule; or choose the smallest ε whose step $\sqrt{\varepsilon}$ exceeds the typical spacing of the training points, a sampling floor.

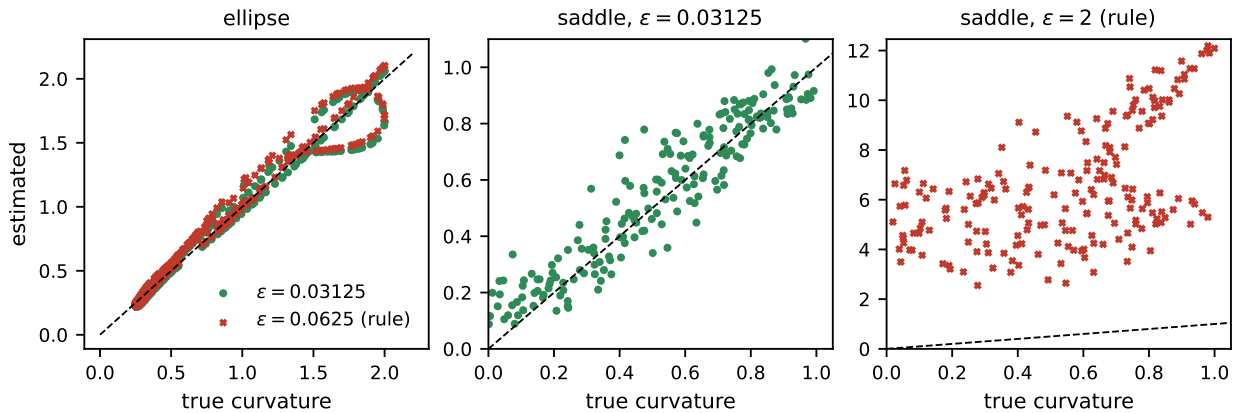


Figure 8: Estimated against true curvature, point by point (2000 training points, $\sigma = 0.02$). *Left:* the ellipse is tracked closely across its eightfold range, with some under-reading at the sharpest bends. *Middle:* the saddle at a sensible noise level. *Right:* the same saddle at the noise level chosen by the automatic rule; note the vertical scale.

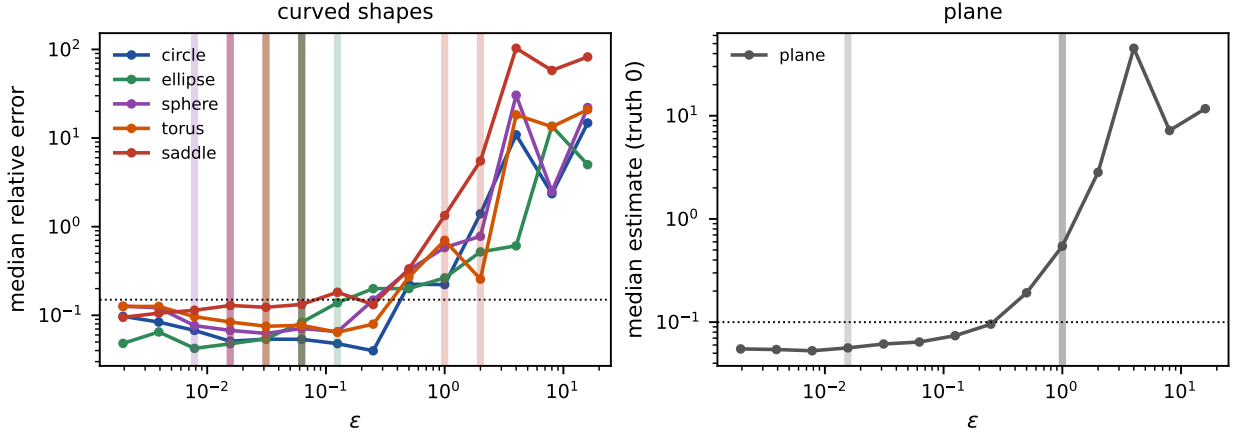


Figure 9: Curvature error against the noise level ϵ at which the networks are used (2000 points, $\sigma = 0.02$; dotted lines: the targets we set in advance). The experiment was repeated three times with different random seeds: different sampled points, noise, rotation, network initialisation and test directions. Each curve is the median over the three runs. The shaded vertical bands mark the noise level the automatic rule chose in each run, one band per seed; overlapping bands look darker. On the saddle the rule chose 0.016, 1 and 2 in the three runs, and on the plane 0.016, 1 and 1. So the rule is not only biased upwards on near-flat shapes but also unstable from run to run. On the curved shapes its choices stay inside the plateau (between 0.008 and 0.125). All curved shapes sit below the target over a wide plateau ($\epsilon \approx 0.002$ –0.25). The plane’s estimate stays near zero over the same range. The automatic rule picks ϵ far too large on the saddle and the plane.

What this could mean for anomaly detection. What we have shown is narrow: on synthetic shapes, and at a sensible noise level, the same two networks that give the anomaly score can also measure the curvature of the healthy set. None of what follows has been tested on time series. It is how we expect curvature to matter, written down so it can be tested.

- **Where the score can be trusted.** The score uses the tangent plane at the denoised point $m(y)$. That is only meaningful if y has a single nearest point on the healthy set. On the curved side of a set, that stops being true at about the radius of curvature, $1/\kappa$, from the set: beyond it, two parts of the set are about equally near. This is exactly the “inside the bowl” region where the denoiser fails (Figure 3) and where the two-regimes case behaves differently. The largest distance at which the nearest point is always unique is called the *reach* of the set, and it is at most $1/\kappa_{\max}$. Estimated curvature would therefore tell us, window by window, whether the score is in its reliable range.
- **Correcting a curvature bias in healthy scores.** The denoiser pulls points towards the inside of a curved set by about $\frac{\epsilon}{2}$ times the mean curvature. So, on a strongly curved stretch of the healthy set, even healthy windows get a non-zero denoising step, and could raise false alarms there. Knowing the local curvature would let us subtract that expected healthy step before scoring.
- **Curvature as a signal in itself.** Some anomalies change the dynamics without leaving the range of healthy values, like the Lorenz parameter change. These may show up as a change in the local shape of the set of windows, so comparing the curvature at test windows with the curvature learned from training windows is a possible detector.

Two cautions apply. First, measuring curvature on real delay windows has not worked so far: our earlier kernel-based estimate resolved curvature on only 1 of 27 benchmark series. Second, the automatic choice of ϵ must be fixed first. The next step on this thread is therefore delay windows with known geometry: windows of a pure sinusoid trace an ellipse, and windows of two tones trace a flat torus. Real series come after that.

6 The learned score on synthetic time series

The learned score. By “the learned score” we mean the score of Section 3, computed from the two networks:

$$s_2(y) = \|(I - 2\Gamma(z))(y - m(y))\|, \quad z = y \text{ or } z = m(y).$$

It is “learned” in contrast to the two earlier detectors, the global and local detectors of Section 1, which are closed-form and involve no training. We compare both placements of the projector: $z = y$, the version used in our first experiments, and $z = m(y)$, the geometric version of Section 2. For each series:

1. cut the series into overlapping windows of length $L = 64$, normalised with the mean and standard deviation of the healthy training part, and scaled by $1/\sqrt{L}$;
2. train the denoiser m and the metric network Γ on the training windows only (rank 16, noise levels sampled between 10^{-4} and 16, 3000 epochs);
3. score every window with s_2 at $\varepsilon = 0.01$, a value fixed earlier on these same cases;
4. spread each window’s score over the time points it covers, with the same smoothing as the earlier detectors, and evaluate with VUS-PR.

We also report the raw denoising step $\|y - m(y)\|$, the score with no projector at all.

The test. We tested the learned score on six synthetic time series built so that we know which of the earlier detectors should win (three random seeds each; each series is described and drawn in Appendix B). Before running, we wrote down three predictions. The two placements of the projector, at y and at $m(y)$, would agree within 0.05 on anomalies inside the span. On the one case where the learned score had disappointed, a fast foreign oscillation added to a Lorenz series, the placement at $m(y)$ would help only if the denoiser actually pulls the anomalous windows back to the healthy set.

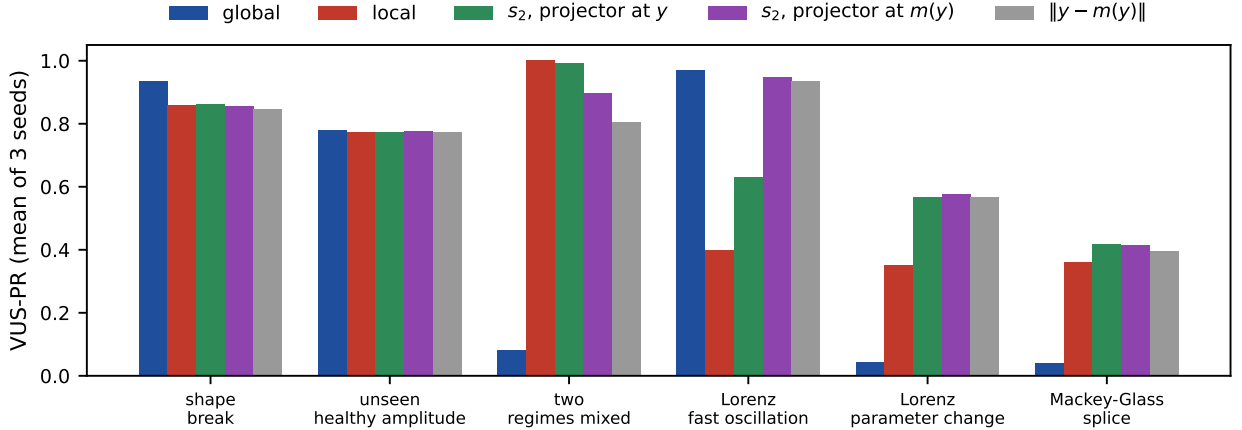


Figure 10: VUS-PR on six synthetic cases (mean of three seeds, $\varepsilon = 0.01$). The global detector fails on the three cases whose anomalies stay inside the span (two regimes, Lorenz parameter change, Mackey–Glass). The local detector fails on the Lorenz fast oscillation. The learned score with the projector at $m(y)$ (purple) is the only score within 0.10 of the best on every case.

Figure 10 gives the results. The denoiser does pull the Lorenz anomalies back: the denoising step is, at the median, 98% of the distance to the nearest healthy window. Moving the projector to $m(y)$ lifts that case from 0.63 to 0.95, close to the global detector’s 0.97. The prediction for the inside-the-span cases held on two of three cases but failed on the two-regimes case, where the projector at $m(y)$ is worse (0.90 against 0.99). Averaged over the six cases, the projector at $m(y)$

scores 0.745, against 0.707 at y and 0.723 for the local detector. With three seeds per case, that ranking is indicative, not established.

Figures 11 and 12 show why the two placements differ. Looking closely, the score with the projector at y carries a second signal besides the distance from the set. Where a window could have come from two distant parts of the healthy set, the network is very unsure of the healthy window behind it, and $2\Gamma(y)$ has eigenvalues *above* 1. The factor $I - 2\Gamma$ then amplifies the denoising step instead of trimming it. Between two healthy regimes (Figure 11; compare probe B in Figure 3) this flags the anomaly. On the Lorenz attractor (Figure 12) it also flags some *healthy* windows, which then outrank the real anomalies. On the anomalous windows themselves the two placements score the same. Evaluated at $m(y)$, the projector loses most of this ambiguity signal and keeps mainly the distance. This explanation was worked out after seeing the results, from the network’s eigenvalues and the per-window scores. It has not yet been tested as a prediction.

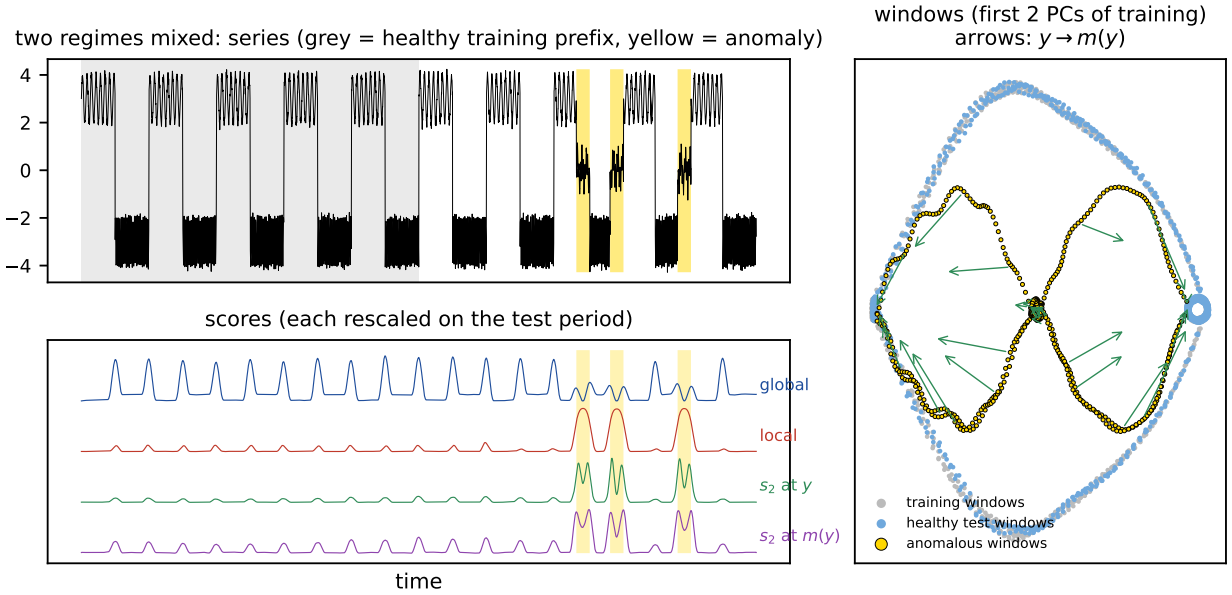


Figure 11: Two healthy regimes, with anomalies that mix them. *Left:* the series and the four scores over time. *Right:* the windows projected onto their first two principal components. The anomalous windows (yellow) lie inside the loop formed by healthy windows: inside the span, off the set. The arrows show where the denoiser sends them.

A dead spot in the middle of the two-regimes anomalies. In Figure 11, the denoiser-based scores dip in the middle of each anomaly. Figure 13 shows why, by following a window as it slides across an anomaly.

- **Where the dip happens.** Windows that lie entirely inside the anomaly are the *farthest* from every healthy window (0.93). Yet the denoiser moves them the least: about 0.12, only 12–15% of that distance.
- **Why.** The anomaly is built as the exact average of the two regimes, so such a window is the midpoint between a regime-A window and a regime-B window. The denoiser returns a best guess that is an average over healthy windows. With equally good candidates on opposite sides, the pulls cancel and the output stays near the input. In the principal-component plot of Figure 11, these windows sit at the centre and barely move. This is the same effect as probe B inside the bowl in Section 2.
- **It is not a training failure.** At $\varepsilon = 0.01$, the noise added during training has a typical size of $\sqrt{0.01 \times 64} \approx 0.8$ in these 64-dimensional windows, so the network has seen inputs about this far from the data. A perfect averaging denoiser would also return the midpoint.

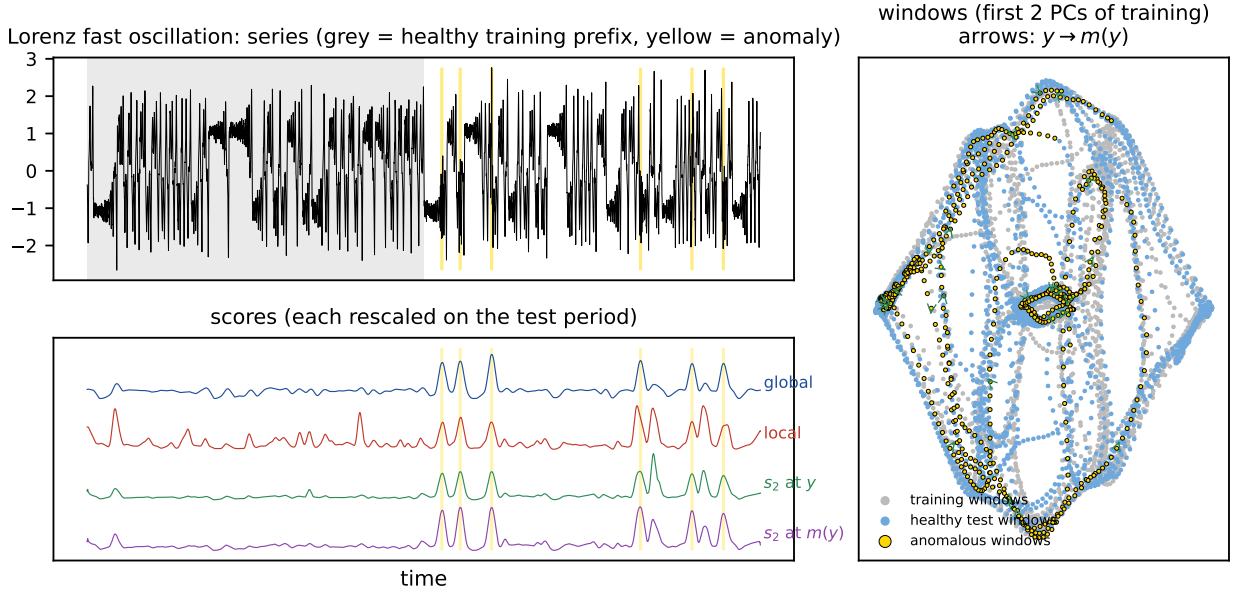


Figure 12: A Lorenz series with a fast foreign oscillation added in short bursts (yellow). With the projector at y (green), some healthy windows score as high as the anomalies. With the projector at $m(y)$ (purple), the anomalies stand out cleanly.

The dead spot comes from what a denoiser computes, an average and not a projection, which would pick one side and move the window the full 0.93.

- **The window length only partly matters.** The dip needs windows lying entirely inside the anomaly. These exist because the anomaly (120 samples) is longer than the window (64). With $L \geq 120$, every window would include some healthy regime and the dip would disappear. But that hides the mechanism rather than removing it: any anomaly that sits midway between two parts of the healthy set lands in the same dead spot.
- **The local detector does not dip.** Its score, the distance to the nearest training window, is largest exactly there.

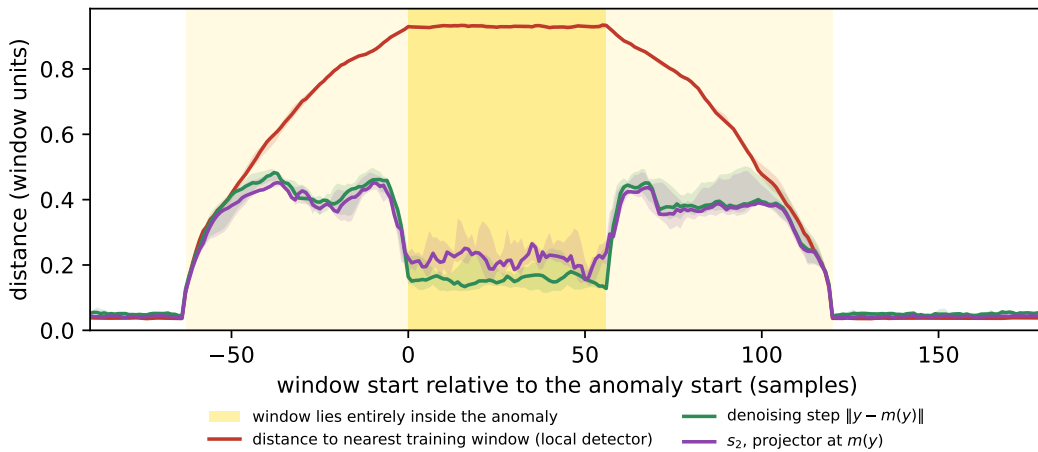


Figure 13: Two-regimes case: a window sliding across an anomaly (all nine anomalies of the three seeds, aligned on the anomaly's start; median and interquartile band; anomaly length 120, window length 64). Light yellow: the window overlaps the anomaly; dark yellow: it lies entirely inside. There, the distance to the nearest healthy window (red) is largest, but the denoising step (green) and the score (purple) collapse.

Options for the dead spot (hypotheses, none tested yet).

1. **A projection-like step: repeated denoising.** Apply the denoiser several times with decreasing ε , as a diffusion model does when it generates samples. Each step moves towards the side that is slightly more likely, so the window ends on one part of the healthy set instead of midway. The exact midpoint is an unstable balance, so small asymmetries are enough to pick a side. Its cost is several forward passes per window.
2. **Use the ambiguity signal directly.** At a midpoint, the healthy windows the denoiser averages over lie on both sides, so their spread is large and $2\Gamma(y)$ has an eigenvalue well above 1. A separate ambiguity term, for example the amount by which the eigenvalues of $2\Gamma(y)$ exceed 1, would flag such windows. This is the split into a distance term and an ambiguity term proposed in the summary.
3. **Combine with the nearest-neighbour distance.** The local detector does not have this dead spot. A combination would inherit its false alarms on new healthy behaviour, so the combination rule would have to be chosen on training data only.
4. **Longer windows.** This removes the dip for anomalies shorter than the window, but dilutes short anomalies and does not fix the mechanism. We list it only for completeness.

7 On the benchmark

We then ran the learned score, unchanged, on the 48 tuning series of the TSB-AD-U benchmark. The window length was the benchmark’s own period estimate on the training part of each series (4 to 288 samples, median 93), and $\varepsilon = 0.01$, fixed beforehand on the synthetic cases. Its mean VUS-PR was 0.465. That is not significantly below our earlier global detector (0.514; difference -0.050 , 95% interval -0.153 to $+0.052$), but the interval is too wide to call the two equal. It is significantly *below* the local detector run on the same windows (0.507; Wilcoxon $p = 0.03$).

Figure 14 shows where it loses. The losses are concentrated at long windows. A likely reason, not yet tested, is that the windows are scaled by $1/\sqrt{L}$. A fixed ε therefore means much more relative noise at long windows than on the synthetic cases ($L = 64$) where it was chosen, and the denoiser then smooths away the detail that matters. The next run scales ε with L .

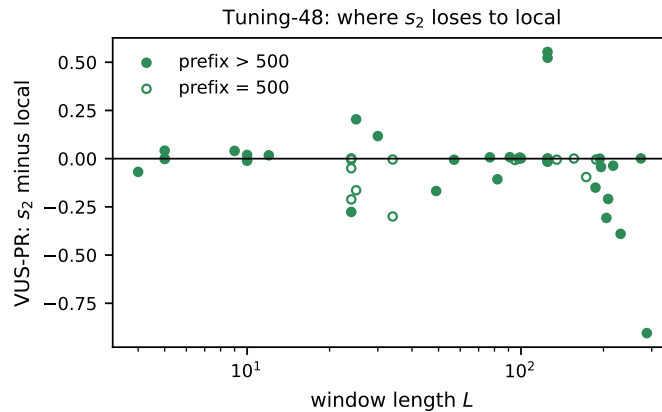


Figure 14: Each point is one benchmark series: the learned score’s VUS-PR minus the local detector’s, against the window length. Open circles have the shortest training prefix (500 samples).

8 Summary and next steps

What we found.

- **The learned networks do what they are meant to, approximately.** The denoiser pulls windows back towards the healthy set. The metric network finds its tangent directions and its dimension, about as accurately as a tuned local PCA on simple shapes.
- **Two systematic departures from the ideal picture.**
 - The denoiser is an average, not a projection. Windows midway between two parts of the healthy set are barely moved: a dead spot.
 - The metric network compresses the range of 2Γ 's eigenvalues: about 3 where the exact value reaches 80 (Appendix A).
- **The networks also measure curvature,** once the noise level is chosen sensibly. The automatic rule for choosing it fails on flat or nearly flat data, and is unstable between runs.
- **Where the projector is evaluated matters.** Evaluating it at the denoised point, the geometric choice, keeps mainly the distance from the healthy set. Evaluating it at the window itself also picks up an ambiguity signal. On the synthetic cases, the first gave the most even performance of any score we have: within 0.10 of the best score on every case.
- **On the real benchmark the learned score is not yet competitive.** It falls well short at long windows, probably because the noise level was not scaled with window length.

Next steps. These are grouped by thread, in the order we would tackle them. Each will have its predictions written down before it is run. Development uses the 48-series Tuning split, and the 350-series evaluation split is used once, on a frozen method.

1. Fix the score's known weaknesses, on the synthetic cases and the 2-D toy.

- *Dead spot:* try repeated denoising with decreasing ε , which should move a midpoint window to one side.
- *Two signals:* split the score into an explicit distance term (projector with eigenvalues clipped to $[0, 1]$, evaluated at $m(y)$) and an explicit ambiguity term (how far the eigenvalues of $2\Gamma(y)$ exceed 1), and see whether each catches the cases it should.
- *Compressed eigenvalues:* find out why the network levels off near 3, for example its rank, its capacity, or too little training at large noise levels. The ambiguity term depends on it.

2. Return to the benchmark (Tuning-48). Rerun with:

- the noise level scaled with window length, $\varepsilon = 0.64/L$, so that the relative noise matches the synthetic cases;
- the projector at $m(y)$;
- cross-fitting on every series, so no training window is scored by a network that saw it;
- a check that each network fits its training data;
- the trained networks saved, so failures can be diagnosed afterwards.

This also tests whether the drift case 560_YAHOO, missed so far, is caught.

3. Curvature.

- Replace the automatic ε rule with a plateau rule (the ε where the estimate changes least) or a sampling floor ($\sqrt{\varepsilon}$ just above the spacing of the training points), and test it on fresh seeds.
- Then move to delay windows with known geometry: a pure sinusoid (an ellipse) and two tones (a flat torus), before any real series.

- Only then ask whether curvature helps detection: as a check on when the score is in its reliable range, as a correction for the inward bias, or as a feature.

4. Tangent planes, from the re-reading of the first experiments.

- Reproduce the Riemannian Metric Matching paper’s own result (an 8-dimensional sphere in $\mathbb{R}^{64}$, against its nearest-neighbour baseline) as a check of our implementation.
- Try a noise level chosen point by point, where a single global value failed near junctions and in sparse regions.
- Redo the real-data consistency test without its confounds: overlapping windows, an assumed dimension, and the old window-length rule.

5. Window length. Revisit choosing L from the geometry of the healthy windows, with the tangent dimension estimated point by point rather than assumed.

A What the metric network returns away from the healthy set

For the 2-D toy, the quantity the network is trained to match can be computed exactly from the 1000 training points. It is the scatter, divided by ε , of the training points weighted by how likely each one is to have produced y under the added noise. On the set this is the tangent projector: eigenvalue about 1 in the tangent direction and about 0 in the orthogonal one. It is not exactly 0, because the training arc has some thickness. Off the set, the larger eigenvalue rises above 1 inside the bowl, where two sides of the arc are about equally near. It falls below 1 outside the bowl and past the arc’s end.

Figures 15 and 16 compare the network with this exact answer. The network gets the directions and the overall pattern right: 0.95 against an exact 1.08 at the bottom of the arc, above 1 inside the bowl, below 1 outside. But it compresses the range. It levels off near 3 where the exact value climbs to about 80 at A, which is equally close to both ends of the arc, and it over-estimates small values far from the data (0.4 against 0.05 well below the arc). The thin red rays far from the data in the exact map reflect which individual training points happen to be nearest, not the shape of the arc; the network’s smoother map is arguably the more useful one there. Evaluated at the denoised point, $2\Gamma(m(y))$ stays between about 1 and 2, as expected for a point on or near the set.

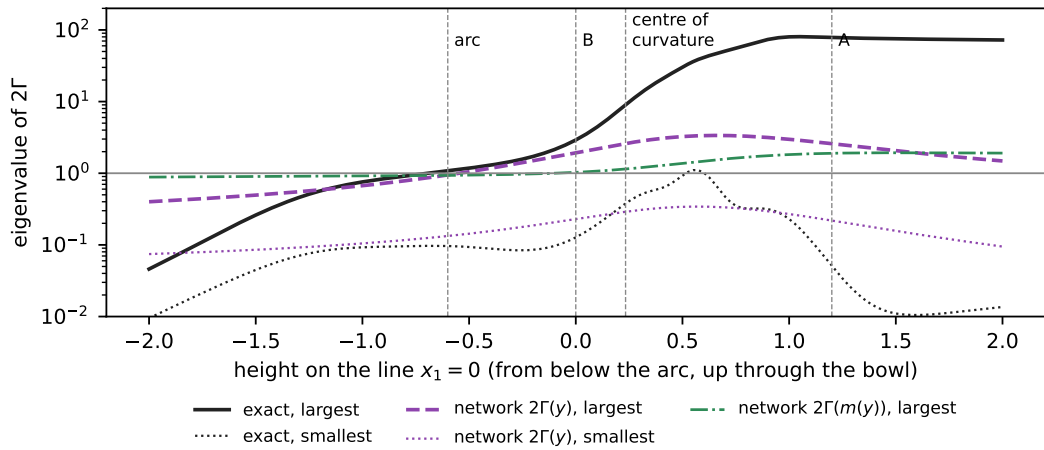


Figure 15: Eigenvalues of 2Γ along the vertical line $x_1 = 0$, from below the arc, through its lowest point, up through the bowl (log scale; the horizontal line marks 1). Near the arc the largest eigenvalue belongs to the tangent direction and the smallest to the orthogonal one.

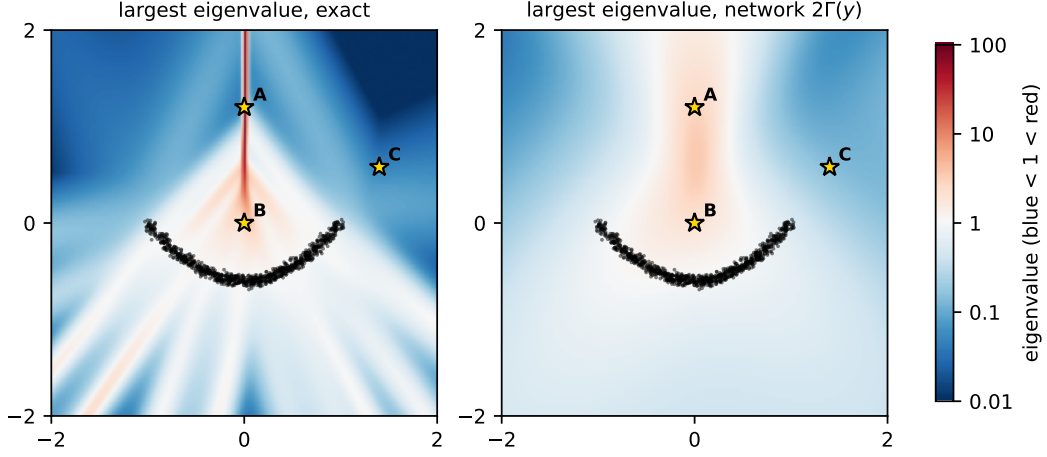


Figure 16: The largest eigenvalue of $2\Gamma(y)$ over the plane, exact (left) and network (right), on a colour scale centred on 1: blue below 1, white at 1, red above.

B The toy datasets in detail

Every toy below was generated by the experiment code named in brackets; Figures 17 and 18 show them. Except for the 2-D toy, each shape is built in its own low dimension, placed in $\mathbb{R}^{20}$ by a random rotation, and then given independent Gaussian noise of standard deviation σ in every one of the 20 coordinates. Each setting was repeated with two or three *random seeds*. A seed is the starting value of the random number generator, so each seed gives a fresh draw of the sampled points, the noise, the rotation and the networks' initial weights. Results are reported as medians or means over seeds, and the spread between seeds shows how much a result depends on chance.

The 2-D toy (Sections 1–3; `compute_2d_toy.py`). 1000 points on the arc $x_2 = 0.6x_1^2 - 0.6$, x_1 uniform on $[-1, 1]$, with noise $\sigma = 0.03$. Probe points: $A = (0, 1.2)$, $B = (0, 0)$, $C = (1.4, 0.58)$, the last on the arc's continuation.

Simple shapes for tangent planes (Section 4, left panel; `run_faithful.py`). A circle of radius 1 (dimension 1); a sphere of radius 1 (dimension 2); a flat torus, two unit circles at right angles in four dimensions (dimension 2). Points uniform on each; $n = 500$ or 2000 training points; $\sigma \in \{0, 0.02, 0.05\}$; two seeds.

Harder shapes for tangent planes (Section 4, right panel; `run_complex.py`). 2000 training points each, $\sigma \in \{0.02, 0.05\}$, two seeds.

- **Crossing planes:** two squares $[-1, 1]^2$ that share one axis and meet along it at 60° ; of the 2000 training points, 1200 lie on one square and 800 on the other. Hard region: within 0.2 of the shared line (19% of points).
- **Plane crossed by a line:** points uniform on a flat square $[-1, 1]^2$ (dimension 2) and on a segment of length 2 through the square's centre, at right angles to it (dimension 1); of the 2000 training points, 1680 lie on the square and 320 on the segment. Hard region: within 0.2 of the crossing point, where the dimension changes (5.5% of points, about 110).
- **Swiss roll:** a flat strip of width 1 rolled into a spiral of one and a half turns, with neighbouring layers about 0.3 apart. Hard region: every point, since every point lies near another layer.
- **Unevenly sampled sphere:** a unit sphere with point density proportional to $e^{2(z-1)}$, so the south pole is about 55 times sparser than the north pole. Hard region: the southern hemisphere (13% of points).

Shapes for curvature (Section 5; `run_shapes.py`). $n = 500$ or 2000 ; $\sigma \in \{0, 0.02, 0.05\}$; three seeds. Normal curvature is known exactly for each.

- **circle** and **sphere**, radius 1: curvature 1 everywhere;
- **ellipse** $(2 \cos t, \sin t)$: curvature from 0.25 (flat ends of the minor axis) to 2 (sharp ends of the major axis);
- **flat torus**: curvature between 0.71 and 1 depending on direction;
- **saddle** $z = (a^2 - b^2)/2$ over the unit disc: curvature between 0 and 1, depending on position and direction; evaluated only where $\sqrt{a^2 + b^2} \leq 0.7$, away from the edge;
- **flat square patch** $[-1, 1]^2$: curvature 0; evaluated in the same inner region.

Synthetic time series (Section 6; `Hankel_AD why_local_global_synthetic.py`). Each series has 3000 healthy training samples followed by 3000 test samples, with observation noise of standard deviation 0.1, and is cut into windows of length $L = 64$.

- **Shape break**: the sum of two sine waves (periods 40 and 17 samples). Six anomalies, each a burst of 8 samples shifted up or down by 2.5. The bursts leave the span of healthy windows.
- **Unseen healthy amplitude**: the same signal, with amplitude varying between 0.8 and 1.2 during training. In the test part, 1000 samples run at amplitude 1.6; they are healthy and not labelled, but were never seen. Five spike bursts follow as the labelled anomalies. This tests false alarms on healthy but new behaviour.
- **Two regimes mixed**: the series switches every 300 samples between regime A (level +3, sine of period 40) and regime B (level -3, sine of period 13). Three anomalies of 120 samples are the average of A and B: both frequencies at once, at level 0. They lie inside the span of healthy windows but away from both regimes.
- **Lorenz fast oscillation**: the first coordinate of the chaotic Lorenz system, rescaled. Six anomalies of 30 samples add a sine wave of amplitude 0.8 and period 7, faster than anything in the healthy series, so they leave the span.
- **Lorenz parameter change**: the same system, but for six stretches of 30 samples one parameter changes (ρ from 28 to 20) and then returns. The dynamics change while the values stay in the healthy range.
- **Mackey–Glass splice**: the Mackey–Glass delay equation with the MGAB benchmark’s parameters. Each of six anomalies cuts the series and rejoins it 100–200 samples later, at the point whose value and first three derivatives best match, following the MGAB procedure. The 20 samples around each join are labelled. The join is nearly invisible by eye.

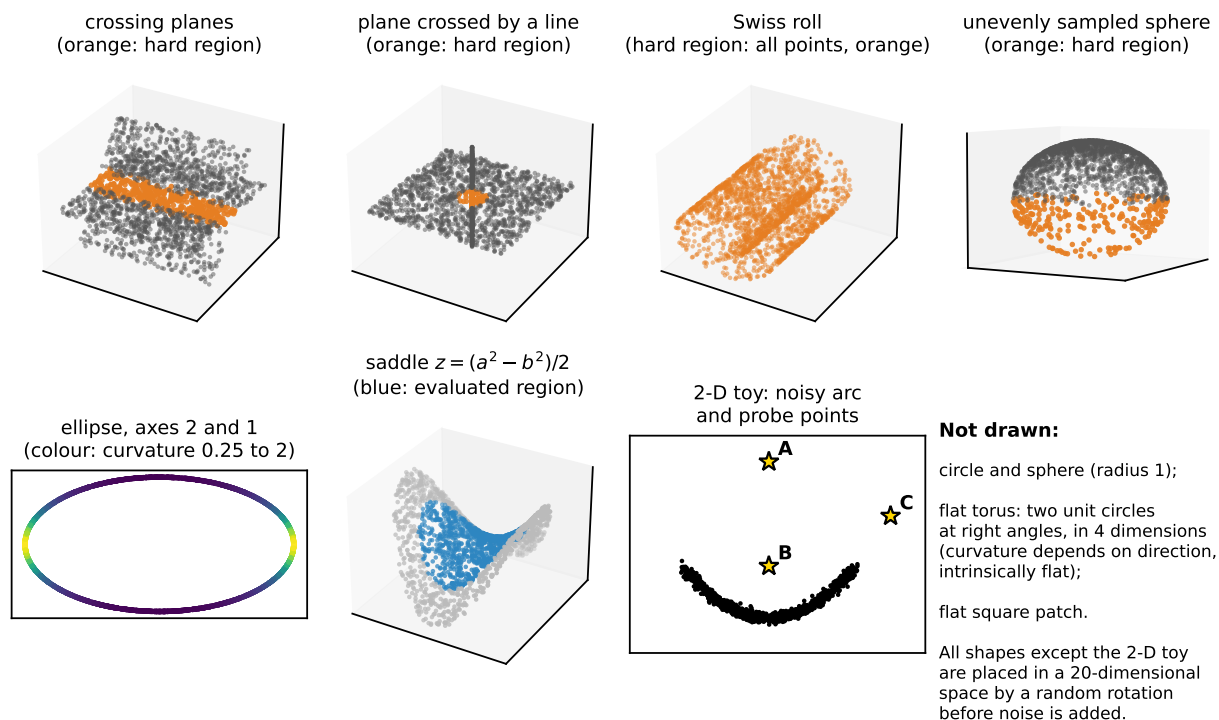


Figure 17: The shapes, drawn in their own three or two dimensions. *Top:* the harder shapes for tangent planes, with each one's hard region in orange. *Bottom:* two curvature shapes (the ellipse coloured by its true curvature; the saddle with the evaluated region in blue) and the 2-D toy.

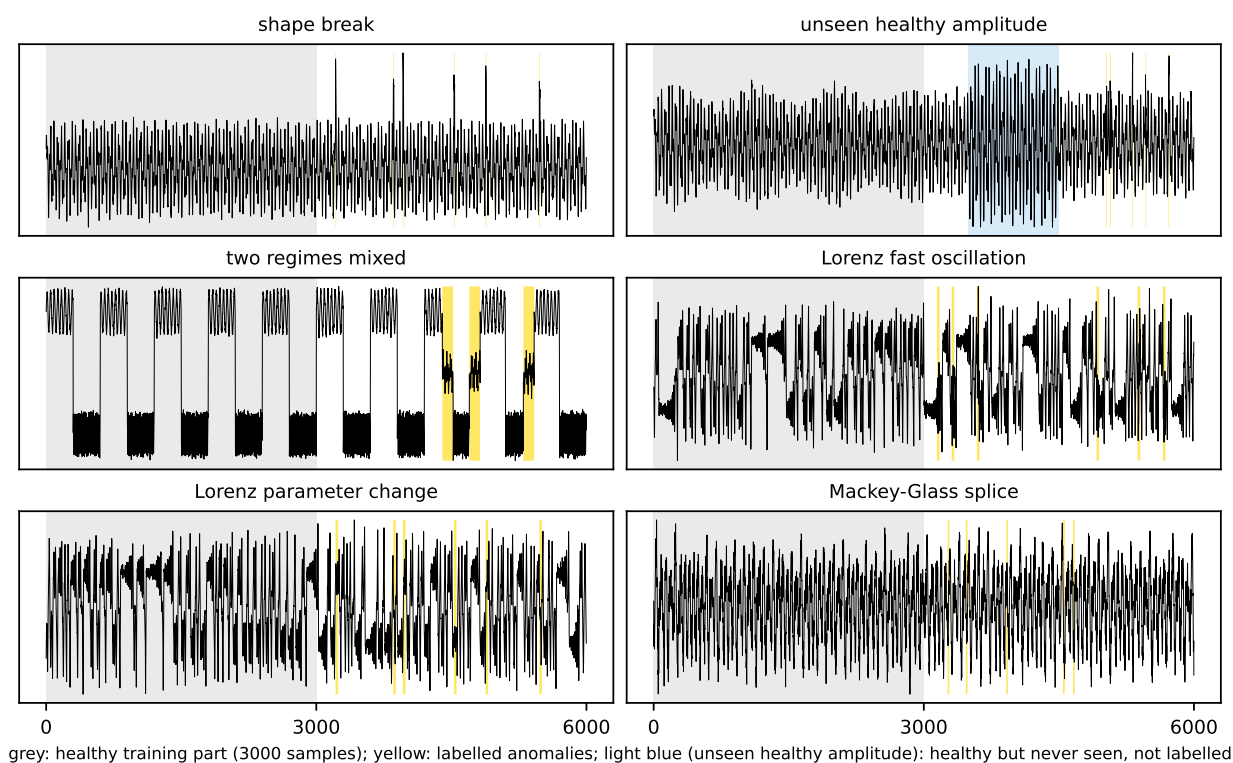


Figure 18: The six synthetic time series (one seed). Grey: the healthy training part; yellow: labelled anomalies (some are only a few samples wide); light blue: healthy behaviour at an amplitude never seen in training, which a good detector should *not* flag.